HTTP ERROR CODES THAT ONE MUST KNOW

When you're building React applications that fetch data from APIs, you'll often come across HTTP status codes in the **Network** tab of your browser's developer tools. These status codes indicate the result of the request: whether it succeeded or failed, and why.

This document explains the most important status codes with simple examples that frontend developers commonly face.

200 — OK

Meaning: The request was successful, and the response contains the requested data. **Example:** You fetched a list of users, and it returned the correct data.

Frontend Tip: Everything is working as expected.

201 — Created

Meaning: A new resource was successfully created (commonly used with POST requests).

Example: You submitted a signup form, and a new user account was created. **Frontend Tip:** You can show a message like "Account created successfully."

204 — No Content

Meaning: The request was successful, but the server returned no content.

Example: A DELETE request completed, but nothing was returned. **Frontend Tip:**Don’t try to parse the response using `.json()` — there’s nothing to parse.

400 — Bad Request

Meaning: The request was invalid or missing some required fields.

Example: Sending an incomplete form or wrong data format to the backend. **Frontend Tip:** Double-check your input and request body.

401 — Unauthorized

Meaning: Authentication is required and has failed or is missing. **Example:** Trying to access a protected route without logging in. **Frontend Tip:** Redirect users to the login page.

403 — Forbidden

Meaning: The server understands the request but refuses to authorize it.

Example: A non-admin user tries to access admin-only resources.

Frontend Tip: Display an "Access Denied" or "Permission Denied" message.

404 — Not Found

Meaning: The requested resource could not be found.

Example: Wrong API endpoint like `/ap/users` instead of `/api/users`. **Frontend Tip:** Show a "Page not found" or "Data not available" message.

409 — Conflict

Meaning: The request conflicts with the current state of the server. **Example:** Trying to register with an email that already exists.

Frontend Tip: Display a clear message to the user like "Email already taken."

429 — Too Many Requests

Meaning: Too many requests have been made in a short period.

Example: Button clicked multiple times very quickly.

Frontend Tip: Add throttling or disable the button temporarily.

500 — Internal Server Error

Meaning: An unexpected error occurred on the server.

Example: Server-side bug or database crash.

Frontend Tip: Nothing is wrong with your code; show a generic error message like "Something went wrong."

502 — Bad Gateway

Meaning: Server received an invalid response from another server. **Example:** The backend server your API depends on is down.

Frontend Tip: Temporary issue — you may retry after some time.

503 — Service Unavailable

Meaning: The server is temporarily unavailable (due to maintenance or overload). **Example:** Backend is restarting or under heavy traffic.

Frontend Tip: Display a message like "Server is busy, please try again later."

504 — Gateway Timeout

Meaning: The server didn't respond in time.

Example: Database query took too long to return results.

Frontend Tip: Suggest the user to retry. Could mean the backend is slow or unresponsive.

Common Real-World Debugging Scenarios

Status Code	Likely Issue	What You Should Check
200	Everything is fine	No action needed
201	Resource created	Confirm action like user registration
204	No content returned	Don't expect data in response
400	Bad request	Validate inputs and request body

401	Not logged in	Redirect to login page
403	No permission	Block action or show error
404	URL is incorrect or resource not found	Check your fetch/axios URL
409	Duplicate data or invalid update	Inform user about conflict
429	Too many clicks/requests	Use throttling or disable buttons
500	Backend/server crashed	Inform backend team, show error
502/503/504	Server/downstream issues	Retry later or show maintenance msg

Bonus (React Example)

```
fetch('/api/data')
 .then((response) => {  if (!response.ok) {  throw new Error(`Request
failed with status  ${response.status}`);
 }
 return response.json();  })
 .then((data) => {  console.log(data);
 })
  .catch((error)  =>  {  console.error('API  Error:',  error.message);
});
```